CS 230 Project Software Design Template
Version 3.0

Table of Contents

Document Revision History

Version	Date	Author	Comments
1.0	**03/12/26**	Darby Crellin	**Initial software design document**
2.0	4.10.26	Darby Crellin	
3.0	4/17/26	Darby Crellin	Recommendations

Instructions
Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

The Gaming Room is developing a web-based game called Draw It or Lose It, which is designed to support multiple users across different platforms. The company's goal is to create a scalable and efficient application that allows players to interact in real time while maintaining strong performance and reliability. To achieve this, the system must support cross-platform functionality, efficient memory and storage management, and a stable server environment.

Requirements

The primary business requirement for Park Station Manufacturing is to produce professional training videos that educate customers on how to properly use their products. These videos will help improve customer experience, product understanding, and overall support.

The main technical requirement is access to high-quality video editing software that is only available on macOS. Since the company currently operates entirely on Windows computers, the solution must allow the organization to use Mac-based video editing tools while still supporting their existing infrastructure.

Design Constraints

One design constraint is that Park Station Manufacturing currently uses only Windows-based computers throughout the organization. Because the video editing software they want to use runs only on macOS, this creates a limitation in how the software can be implemented within the company's current environment.

Another constraint is the requirement to use professional video editing software that is only available on Mac systems. This restricts the available technology options and requires the company to consider introducing Mac hardware or another compatible solution.

A third constraint involves budget and resource considerations. Introducing new hardware, software licenses, and training for employees may increase costs and require careful planning. The company must find a solution that meets their technical needs while remaining financially practical.

System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

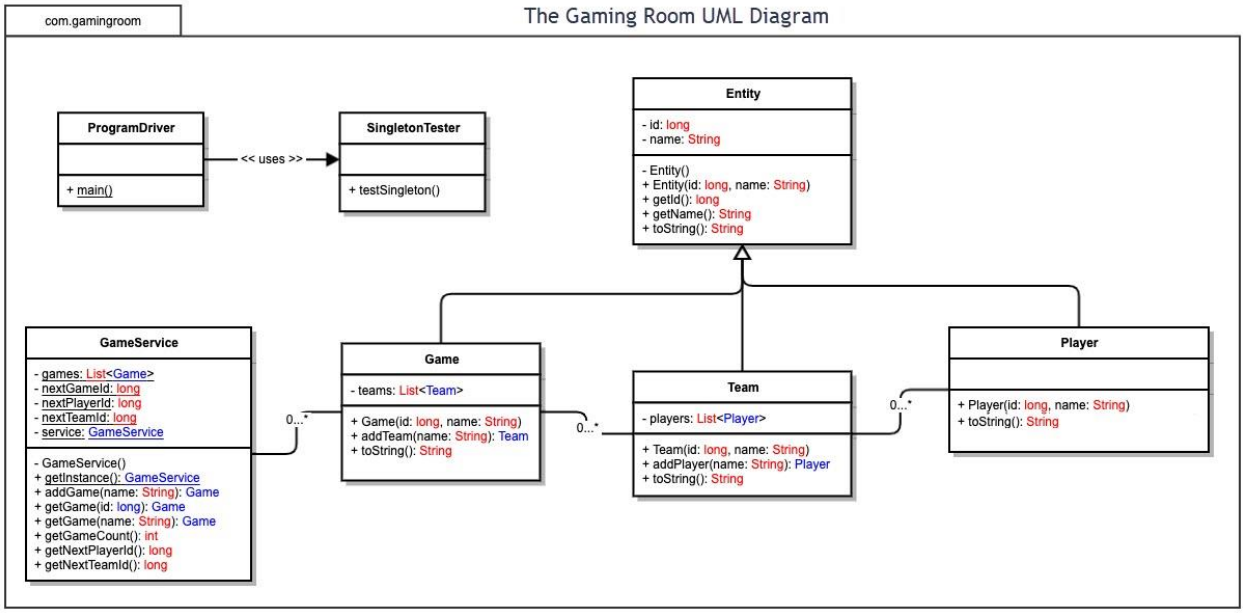
Domain Model

The UML diagram represents the structure of the gaming application and shows how the main classes interact with one another. The central class in the design is the GameService, which acts as a manager for creating and maintaining game instances. It stores a list of games and provides methods for adding, retrieving, and managing game data.

The Game class represents an individual game session. Each game contains a list of teams and includes attributes such as a unique identifier and a name. The relationship between GameService and Game demonstrates aggregation, as the GameService manages multiple Game objects.

The Team class represents groups of players participating in a game. Each team contains multiple players and includes attributes such as a team ID and name. The relationship between Game and Team shows that a game can have multiple teams, illustrating a one-to-many relationship.

The Player class represents individual participants in the game. Each player belongs to a team and has attributes such as a player ID and name. This establishes a one-to-many relationship between Team and Player.
The Entity class serves as a base class that defines common attributes such as ID and name. The Game, Team, and Player classes inherit from Entity, demonstrating the object-oriented principle of inheritance. This allows shared properties to be reused across multiple classes, improving efficiency and reducing redundancy. The diagram includes a SingletonTester and ProgramDriver class. The SingletonTester is used to verify that only one instance of the GameService exists, demonstrating the Singleton design pattern. The ProgramDriver acts as the entry point of the application and interacts with the GameService to run the program.
Overall, the UML diagram demonstrates key object-oriented principles including inheritance, encapsulation, and aggregation. These relationships allow the system to efficiently manage game data while maintaining a clear and organized structure.



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	Mac	Linux	Windows	Mobile Devices

Server Side	Mac systems are not a suitable option for hosting a web-based application in a production environment. Apple discontinued macOS Server and no longer supports server-side hosting capabilities, making it unreliable for commercial deployment. As a result, macOS cannot be effectively used to host and scale a web-based game service that requires consistent uptime and the ability to support thousands of users. While macOS may still be useful for development purposes, it is not a viable solution for server-side hosting in this scenario.	Linux is the most effective platform for hosting a web-based application. It is open-source, which eliminates licensing costs, and it is widely used in enterprise server environments due to its stability and scalability. Linux systems can efficiently handle high traffic and support thousands of users, making it ideal for this application. The main disadvantage is that it may require more technical expertise to manage and configure.	Windows Server is a viable option for hosting web applications and provides strong support for enterprise tools and services. It is user-friendly and integrates well with Microsoft technologies. However, it requires licensing fees and can be more resource-intensive compared to Linux. This may increase operational costs when scaling the application.	Mobile devices are not intended for hosting server-side applications. They lack the processing power, scalability, and reliability required to support a web-based system with many users. Mobile platforms are designed for client-side interaction rather than server-side deployment.

Client Side	Supporting Mac users on the client side requires ensuring compatibility with macOS browsers such as Safari and Chrome. Development considerations include responsive design and cross-browser testing to ensure consistent performance. Costs are moderate since web applications do not require separate Mac-specific development, but testing is still costs are moderate since web applications do not require separate Mac-specific development, but testing is still necessary to ensure consistent performance across browsers.	Linux users can access the application through web browsers such as Firefox or Chrome. Since Linux environments can vary, additional testing may be required to ensure compatibility across distributions. However, development costs remain low because the application is web-based and does not require a separate build.	Windows is the most widely used desktop platform, so ensuring compatibility with browsers such as Chrome, Edge, and Firefox is essential. Development considerations include thorough testing and optimization for performance. Supporting Windows does not significantly increase development costs due to the use of a web-based application.	Mobile devices require responsive design to ensure the application functions properly on smaller screens. Development considerations include touch interface support, performance optimization, and testing across both iOS and Android devices. While this adds time to development, it avoids the need to build separate native applications, reducing overall cost.

Development Tools	Development on Mac systems can use tools such as Visual Studio Code, IntelliJ IDEA, or Eclipse for coding and debugging. macOS is also commonly used for front-end development due to its compatibility with design tools and testing environments. Most tools are free, but some professional software may require licensing.	Linux supports many open-source development tools, including Eclipse, VS Code, and Git. It is highly favored for server-side development and deployment. Since most tools are open-source, development costs are low. However, developers may need more technical experience to work efficiently in a Linux environment.	Windows provides a wide range of development tools such as Visual Studio, Eclipse, and VS Code. It is user-friendly and widely supported, making it accessible for development teams. Some tools, such as Visual Studio, may require licensing, which can increase costs.	Development for mobile compatibility primarily involves web technologies such as HTML, CSS, and JavaScript rather than platform-specific tools. Testing tools and emulators may be used to simulate mobile environments. While additional testing is required, the use of web-based development reduces the need for separate mobile development tools and costs.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform**: For *Draw It or Lose It*, the best choice is a **Linux-based cloud server environment**, such as AWS or Microsoft Azure. Linux is known for being stable, scalable, and cost-effective, which makes it a strong option for hosting a web-based game. It also avoids licensing **costs and is widely used in** real-world server environments, making it reliable for long-term growth. Using a cloud-based platform allows the application to scale as more users join. Instead of being limited to one physical system, resources can be adjusted based on demand, which helps prevent crashes or slow performance during peak usage. This makes it a practical and future-focused solution for the company.

2. **Operating Systems Architectures**: The recommended architecture is a **client-server model within a distributed system**. In this setup, the server manages core functions like game logic, data storage, and user authentication, while the client side handles the user interface. This approach allows the game to run across multiple platforms, including desktops, tablets, and mobile devices, without needing separate versions of the application. It also improves efficiency because updates can be made on the server without requiring users to download anything new. Because the system is distributed, it can also handle higher traffic by spreading workloads across multiple servers. This improves performance and helps keep the game running smoothly even as more players join.

3. **Storage Management**: A **relational database system**, such as MySQL or PostgreSQL, is recommended for managing game data. This allows the application to organize and store structured data like user accounts, game sessions, and scores in a clear and efficient way. Since the game includes a large number of image files, cloud storage should also be used to support scalability and accessibility. Storing data in the cloud ensures that information is available across platforms and protected from hardware failure. Overall, this combination allows for fast data retrieval, better organization, and the ability to grow the application over time without major system changes.

4. **Memory Management**: The system should use **efficient memory management techniques** to keep performance smooth during gameplay. Instead of loading all game assets at once, techniques like lazy loading should be used so that images are only loaded when needed. This helps prevent unnecessary memory usage, especially on devices with limited resources. Caching is also important because it allows frequently used data to be stored temporarily for faster access. This reduces load times and improves the overall user experience. Modern operating systems like Linux also help manage memory automatically by allocating and freeing resources as needed. This ensures that the application remains stable and responsive during extended use.

5. **Distributed Systems and Networks**: To support communication across multiple platforms, the game should operate as a distributed system using standard web protocols like HTTP and HTTPS.

This allows users on different devices and operating systems to interact with the same application in real time. Load balancing should be used to distribute traffic evenly across servers, which helps prevent system overload and improves reliability. In addition, backup systems and redundancy should be in place to reduce the impact of outages or connectivity issues. By using a distributed network approach, the application can scale effectively while maintaining consistent performance for all users.

6. **Security**: Security is a critical part of the system, especially when handling user data. The application should use HTTPS encryption to protect data being transferred between users and the server. User authentication should include secure login processes and password hashing to prevent unauthorized access. Role-based access control can also be used to ensure users only have access to appropriate features and data. Regular system updates and monitoring should be implemented to identify and address potential vulnerabilities. By prioritizing security, the application protects user information and builds trust with its users.